

Lógica de Programação

Prof. Rodrigo M. M. Lyra





Array

Arrays: coleções fixas de elementos do mesmo tipo.

- Listas (ArrayLists e LinkedLists): coleções dinâmicas de elementos do mesmo tipo.

Os arrays são coleções estáticas de elementos do mesmo tipo. Eles têm um tamanho fixo, o que significa que você deve especificar o número de elementos no momento da sua criação. Cada elemento é identificado por um índice, representado por um número inteiro, que começa em 0 (zero),





Array

Dessa forma, os arrays oferecem acesso rápido aos elementos por meio desses índices e são eficientes em termos de memória. Vejamos um exemplo de como declarar um array chamado nomes em Java e inicializá-lo com cinco elementos do tipo String individualmente:

```
String[] nomes = new String[5];  
nomes[0] = "Ana";  
nomes[1] = "Bruno";  
nomes[2] = "Carlos";  
nomes[3] = "Daniel";  
nomes[4] = "Eduardo";
```

Neste exemplo, estamos declarando um array chamado nomes com capacidade para armazenar 5 (cinco) valores do tipo String (texto). Em seguida, atribuímos um valor a cada elemento desse array, referenciando-o pelo seu índice no array.



Array

Assim como nos demais tipos de variáveis em Java, também é possível inicializar um array no momento da sua declaração. Por exemplo, podemos declarar um array notas composto por cinco valores do tipo inteiro:

```
int[] notas = { 55, 95, 80, 45, 100 };
```

Também podemos acessar os valores dos elementos de um array usando o seu índice, que atribui o valor do segundo elemento do array à variável segundaNota:

```
int segundaNota = notas[1];
```

Para acessar o tamanho do array após a sua inicialização, isto é, o seu número de elementos, devemos utilizar o atributo `length` (que significa “comprimento”). O exemplo a seguir imprime o tamanho dos arrays `nomes` e `notas`.

```
System.out.println("Qtd. Nomes: " + nomes.length);
```

```
System.out.println("Qtd. Notas: " + notas.length);
```



Array

Os arrays em Java podem ser de tipos primitivos, como `int` e `double`, ou de tipos não-primitivos, como `String` ou outras classes. Eles são muito úteis para armazenar coleções de valores relacionados e são amplamente usados em programação. Contudo, por terem um tamanho fixo definido na sua declaração, o tamanho do array não pode ser alterado posteriormente.

Caso tentarmos acessar um elemento com índice maior ou igual ao comprimento do array, a execução do programa será interrompida pois irá lançar uma exceção. De forma simplificada, uma exceção é um evento que ocorre durante a execução de um programa, indicando uma situação excepcional ou um erro.

Ao tentarmos adicionar um novo elemento ao array `nomes`:

```
nomes[5] = "Felipe";
```

A saída será:

```
ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
```



Array

Além disso, os elementos de um array podem ser facilmente percorridos usando loops como for-each ou iterações tradicionais. Isso é bastante útil para processar todos os elementos da lista. O loop for-each (também conhecido como enhanced for loop) é usado para iterar através de todos os elementos do array sem a necessidade de índices.

Por exemplo, para imprimir todos os nomes contidos no array nomes, poderíamos fazer da seguinte forma:

```
for (int i = 0; i < nomes.length; i++) {  
    String nome = nomes[i];  
    System.out.println(nome);  
}
```



ArrayLists

Os ArrayLists, por sua vez, são estruturas de dados dinâmicas que podem crescer ou diminuir conforme necessário. Isso as torna ideais para lidar com coleções de elementos de tamanho variável.

Elas são mais flexíveis do que os arrays e oferecem uma variedade de operações de manipulação, como adição, remoção e pesquisa de elementos.

Considere o exemplo a seguir:

```
import java.util.ArrayList;
public class Listas {
    public static void main(String[] args) {
        ArrayList<String> nomes = new ArrayList<>();
        nomes.add("Ana");
        nomes.add("Bruno");
        nomes.add("Carlos");
        nomes.add("Daniel");
        nomes.add("Eduardo");
        nomes.remove(0);
    }
}
```



ArrayLists

Neste exemplo, estamos criando um ArrayList que armazena elementos do tipo String. Os genéricos em Java, representados pelos símbolos “< >” permitem que você especifique o tipo de dados que a lista irá conter. Isso garante a segurança de tipo, evitando que tipos não desejados sejam inseridos na lista.

Os ArrayLists oferecem métodos convenientes para adicionar, remover e modificar elementos. Podemos usar o método add() para adicionar elementos à lista e remove() para remover elementos da lista. No nosso exemplo, estamos adicionando cinco nomes à lista, enquanto que o método remove(0) remove o primeiro elemento da lista (Ana).

Assim como nos arrays, os ArrayLists permitem o acesso rápido aos elementos por meio dos seus índices. O método get() é usado para acessar um elemento na lista com base em sua posição, definida pelo seu índice. Por exemplo:

```
String segundoNome = listaDeNomes.get(1);  
System.out.println(segundoNome);
```




ArrayLists

Enquanto isso, o método `set` é usado para modificar um elemento na lista com base em sua posição. Por exemplo:

```
nomes.set(2, "Duda");
```

Neste caso, o método `set` substitui o valor presente na posição 2 da lista `nomes` por `Duda`. Vale lembrar que os índices em uma lista começam em 0, então a posição 2 corresponde ao terceiro elemento da lista. Esse método é útil quando você precisa atualizar o valor de um elemento específico sem alterar o tamanho da lista.



ArrayLists

Métodos para lista

Método (Classe Arrays)	Explicação	Exemplo de Uso
<code>Arrays.toString()</code>	Converte um array em uma string para fácil visualização, como "[elemento1, elemento2]".	<pre>int[] arr = {1, 2, 3}; Arrays.toString(arr); // Retorna "[1, 2, 3]"</pre>
<code>Arrays.sort()</code>	Ordena os elementos de um array em ordem crescente.	<pre>int[] numeros = {30, 10, 20}; Arrays.sort(numeros); // numeros agora é {10, 20, 30}</pre>
<code>Arrays.equals()</code>	Compara o conteúdo de dois arrays, retornando true se eles têm o mesmo tamanho e os mesmos elementos na mesma ordem.	<pre>int[] a = {1, 2}; int[] b = {1, 2}; Arrays.equals(a, b); // Retorna true</pre>



ArrayLists

Analogamente aos arrays, os elementos de ArrayLists podem ser facilmente percorridos usando for-each ou iterações tradicionais.

Por exemplo, para imprimir todos os nomes na lista nomes, podemos fazer da seguinte forma:

```
for (int i = 0; i < nomes.size(); i++) {  
    String nome = nomes.get(i);  
    System.out.println(nome);  
}
```

Neste exemplo, usamos um loop for tradicional: inicializamos uma variável *i* com zero, que atuará como um índice/contador; o loop continua enquanto *i* for menor que o tamanho da lista, que é obtido usando o método `size()`; a cada iteração, usamos o método `get(i)` para acessar o elemento na posição *i* da lista; e por fim, imprimimos o elemento na tela.